

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Информация о компании Норбит	6
1.1 Общая характеристика организации	6
1.2 Направления деятельности и связь с фронтенд-разработкой	6
2 Основы фронтенд-разработки.....	8
2.1 HTML и семантическая разметка	8
2.2 CSS, адаптивность и методология BEM	8
2.3 JavaScript, DOM и событийная модель	8
2.4 Браузерные API и управление состоянием	9
3 Выполненные практические задания	10
3.1 Вёрстка лендинга Powered Media	10
3.2 Генератор паролей.....	15
3.3 Игра «Блэкджек».....	16
3.4 Разделитель тысяч.....	17
3.5 Генератор случайных цветов фона.....	18
3.6 Список задач (To-Do List).....	19
3.7 Подбор цветов (колоночная палитра).....	20
4 Результаты практики	22
4.1 Итоги выполнения заданий.....	22
4.2 Освоенные навыки и методические замечания.....	22
4.3 Сравнительный анализ проектов	23
ЗАКЛЮЧЕНИЕ.....	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	25
ПРИЛОЖЕНИЕ А Листинги ключевых фрагментов кода.....	27

ВВЕДЕНИЕ

Производственная технологическая (проектно-технологическая) практика проходила в организации «Норбит» в г. Санкт-Петербурге с 04 августа по 28 августа 2026 года. Практика была направлена на закрепление знаний по направлению подготовки 09.03.02 «Информационные системы и технологии» (направленность «Информационные системы и технологии в бизнесе») и приобретение практических навыков фронтенд-разработки веб-интерфейсов.

Целью практики являлось изучение учебных материалов по теме «Фронтенд-разработка: в мире веб-технологий» и выполнение комплекса практических заданий по созданию веб-страниц и клиентских приложений на языке JavaScript. Для достижения цели были изучены основы HTML, CSS и JavaScript, рассмотрены принципы адаптивной вёрстки и методология BEM, освоены приёмы работы с Document Object Model (DOM), событиями и браузерными программными интерфейсами, а также реализованы семь учебных проектов различной сложности.

Объектом работы выступают клиентские веб-приложения и пользовательские интерфейсы, разрабатываемые средствами HTML5, CSS3 и JavaScript. Предметом работы являются методы построения структуры страницы, стилизации интерфейса, организации клиентской логики и хранения состояния в браузере. Практическая значимость результата заключается в формировании набора работающих приложений, демонстрирующих освоение базового цикла фронтенд-разработки — от статической вёрстки до интерактивных утилит с сохранением данных.

В ходе практики последовательно выполнялись задания: вёрстка многосекционного лендинга с применением Bootstrap и BEM; разработка генератора паролей; реализация игры «Блэкджек»; алгоритм форматирования чисел; демонстрация смены цвета фона; список задач с localStorage; колоночная палитра цветов. Все задания, за исключением разделителя тысяч (оценка 9 баллов), приняты с результатом 10 из 10.

Структура отчёта включает сведения об организации, теоретический обзор, описание заданий с иллюстрациями интерфейсов в основной части, анализ результатов, заключение и список источников. В приложении А приведены листинги ключевых функций с краткими комментариями. Оформление выполнено по ГОСТ 7.32-2017 и правилам ГУАП.

Актуальность темы связана с тем, что веб-интерфейс — основной канал взаимодействия пользователей с корпоративными информационными системами. Поэтому навыки HTML, CSS и JavaScript имеют прикладную ценность для направления «Информационные системы и технологии в бизнесе».

1 Информация о компании Норбит

1.1 Общая характеристика организации

НОРБИТ является российским ИТ-интегратором, специализирующимся на разработке и внедрении цифровых решений для автоматизации процессов в крупном бизнесе и государственном управлении. Компания входит в группу компаний ЛАНИТ и работает как мультивендорный интегратор, одновременно развивая собственные программные продукты. География деятельности охватывает заказчиков из России и стран СНГ; накопленный портфель включает более тысячи реализованных проектов.

Центральный офис расположен в Москве, при этом компания развивает региональную сеть, включая обособленное подразделение в Санкт-Петербурге. Официальный сайт: <https://www.norbit.ru/>.

НОРБИТ ориентируется на комплексную цифровую трансформацию клиентов: от анализа бизнес-процессов до внедрения и сопровождения. В числе продуктов — BPM-платформа NBT, решения SRM, BI, HRM и сервисы лояльности. Качество веб-интерфейсов для таких систем критично.

Практика в профильной компании обеспечивает программу заданий, критерии приёма и обратную связь, что приближает обучение к условиям реальной разработки.

1.2 Направления деятельности и связь с фронтенд-разработкой

К направлениям деятельности относятся CRM, SRM, HRM, ERP, BI и ML, а также системы управления данными и бюджетирования. Значительная часть взаимодействия пользователя с такими системами реализуется через веб-интерфейсы.

Фронтенд в корпоративном контексте включает формы, таблицы, навигацию и сценарии подтверждения действий. Учебные проекты практики отражают те же компетенции: семантика разметки, стилизация, события, валидация ввода, состояние и обратная связь UI.

Программа web-разработки шла от вёрстки и Bootstrap к алгоритмам на JavaScript и приложениям с персистентностью. Это соответствует онбордингу начинающего фронтенд-разработчика.

Таким образом, практика связывает учебные задания с профилем ИТ-интегратора. Далее излагаются теоретические основы и описание выполненных работ.

2 Основы фронтенд-разработки

Фронтенд-разработка — создание клиентской части веб-приложения: структуры документа, оформления и интерактивного поведения в браузере. Стек практики: HTML, CSS и JavaScript; в лендинге дополнительно Bootstrap 5.

Акцент на «ванильном» стеке позволяет понять DOM и события без скрывания механизмов за абстракциями фреймворков. Ниже материал сгруппирован по блокам: разметка, стили, скрипты и браузерные API.

2.1 HTML и семантическая разметка

HTML5 предоставляет семантические элементы `header`, `nav`, `main`, `section`, `footer` и др. В лендинге они использовались для шапки, навигации, секций и подвала.

Формы, `button` и `dialog` обеспечивают стандартное поведение браузера. В генераторе паролей настройки вынесены в `dialog`; в To-Do List добавление задачи идёт через `submit` формы.

Атрибуты `aria-label` повышают доступность кнопок без текста. Для защиты от XSS в To-Do List применяется `escapeHtml` перед вставкой пользовательского текста в DOM.

2.2 CSS, адаптивность и методология BEM

Современная вёрстка опирается на Flexbox, CSS Grid, медиазапросы и CSS-переменные. Методология BEM задаёт имена вида `блок__элемент--модификатор` и снижает конфликты стилей.

Адаптив лендинга вынесен в `media.css` (точки `~768/992/1024 px`): на узких экранах появляется бургер, сетки перестраиваются. Bootstrap ускоряет сетку и кнопки, кастомный CSS задаёт уникальный вид.

Для палитры цветов использован `Grid repeat(5, 1fr)` на `100vh`. Фрагменты CSS лендинга приведены в приложении А.

2.3 JavaScript, DOM и событийная модель

Скрипты находят элементы DOM, меняют содержимое и подписываются на события. Типовая схема: действие пользователя — обновление состояния — render.

ES-модули применены в блэджке (логика в 21.js, UI в index.js). Алгоритмы: генерация паролей/цветов, shuffle Фишера–Йетса, учёт тузов, группировка цифр, фильтрация задач.

Делегирование событий использовано в To-Do List; в блэджке кнопки блокируются по флагу isRoundActive.

2.4 Браузерные API и управление состоянием

Использовались Clipboard API, localStorage, location.hash и setInterval. Состояние удобно хранить в объекте; после изменений вызывается единая отрисовка.

localStorage обеспечивает сохранение задач между сессиями; hash позволяет делиться палитрой ссылкой без сервера. Ключевые реализации — в приложении А.

3 Выполненные практические задания

Практическая часть включала семь проектов репозитория `norbit-internship`. Ниже для каждого задания приведены описание и скриншоты интерфейса. Исходный код ключевых функций вынесен в приложение А.

3.1 Вёрстка лендинга **Powered Media**

Первым крупным заданием стала вёрстка многосекционного маркетингового лендинга **Powered Media**. Макет включает шапку с навигацией, баннер, блоки `about` и ценностей, портфолио, команду, отзывы, тарифные планы, новости, СТА и футер. Цель — структурировать крупную страницу, совместить `Bootstrap` с `BEM` и обеспечить адаптив.

Стек: `HTML5`, `CSS3`, `Bootstrap 5.3`, шрифт `DM Sans`; стили в `styles.css` и `media.css`. Особое внимание — отступам, сеткам карточек и перестроению на мобильных разрешениях. Карточка выделенного тарифа помечена модификатором `plans__card--featured`. Оценка: 10 из 10.

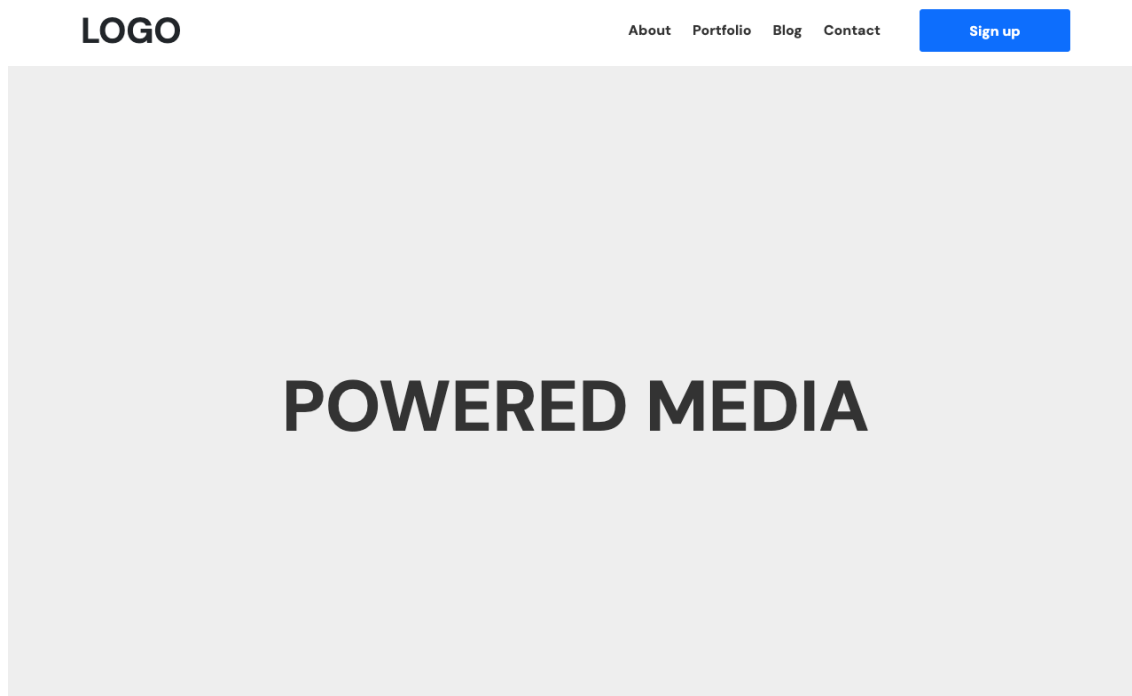
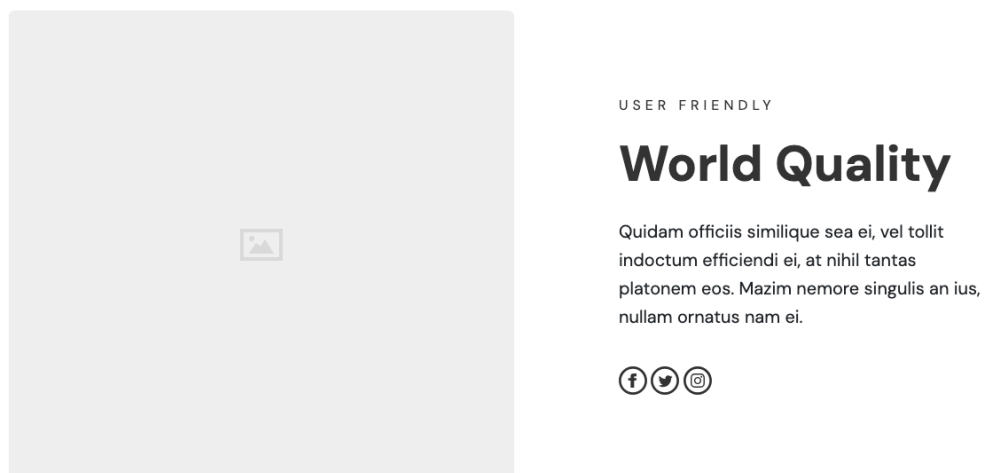
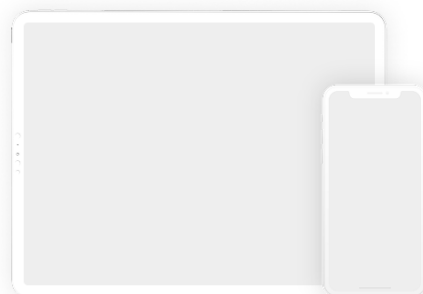
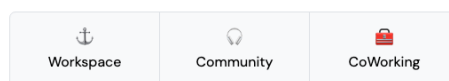


Рисунок 1 — Главный экран лендинга (баннер и шапка)



Company Values

Рисунок 2 — Секция About и ценности компании



Commercial & Personal Use

Quidam officiis similique sea ei, vel tollit inductum efficiendi ei, at nihil tantas platonem eos. Mazim nemore singulis an ius, nullam ornatus nam ei. Dicit euismod invidunt pro, ne his dolorum molestie reprehendunt, quo luptatum evertitur ex. Altera integre suavitate per an, alienum phaedrum te sea.

Portfolio

Рисунок 3 — Блок портфолио

Quidam officiis similique sea ei, vel tollit inductum efficiendi ei, at nihil tantas platonem eos.

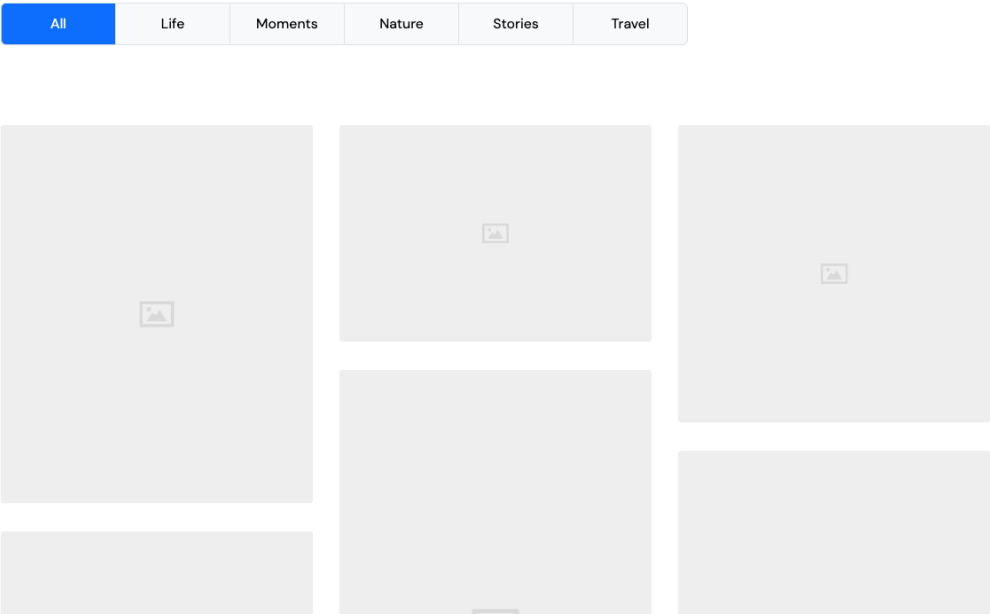


Рисунок 4 — Команда и отзывы

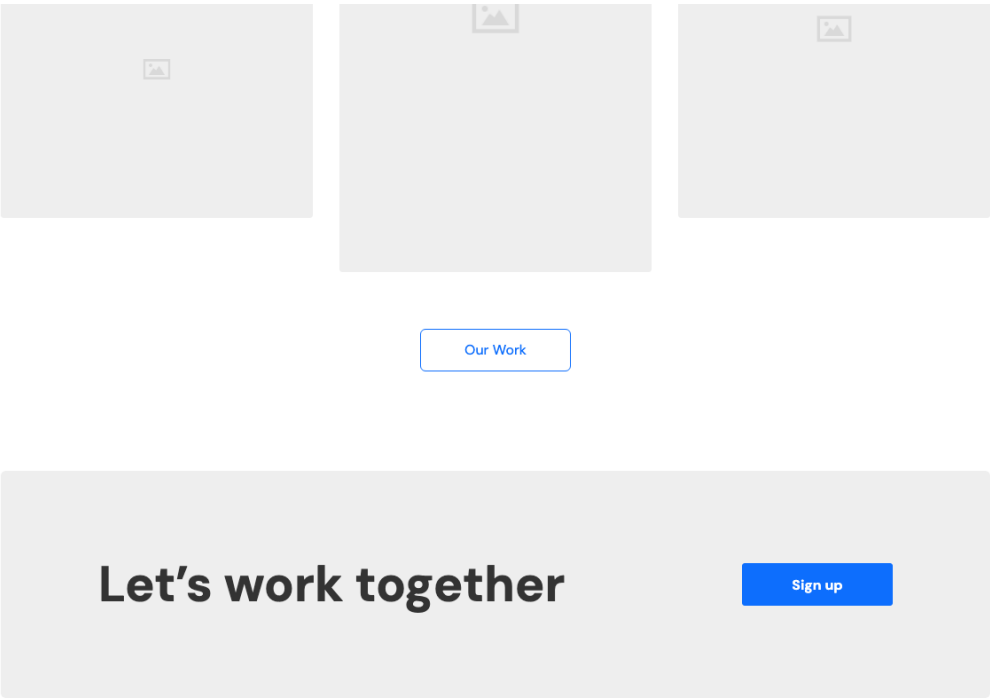
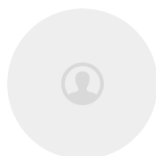


Рисунок 5 — Тарифные планы

Our Team

Quidam officiis similique sea ei, vel tollit indoctum efficiendi
ei, at nihil tantas platonem eos.



Sebastian Bennett

Founder

Ea cum vide impetus deserunt. Percipit eloquentiam sea an, viderer detracto occurreret has id. Mel at
everti ancillae intellegat. Mazim integre an nam. Pro te viderer ancillae scribentur. Inciderint temporibus
id mel. Dolorem maiestatis omittantur id eos. His cu tamquam dolorum veritus.



Рисунок 6 — Нижняя часть страницы и футер

Для проверки адаптивности лендинг также просматривался на ширине мобильного устройства. На рисунках 7 и 8 показаны варианты отображения на узком экране: бургер-меню и вертикальная компоновка секций.

LOGO

Sign up



**POWERED
MEDIA**

Рисунок 7 — Лендинг на мобильной ширине (верх экрана)

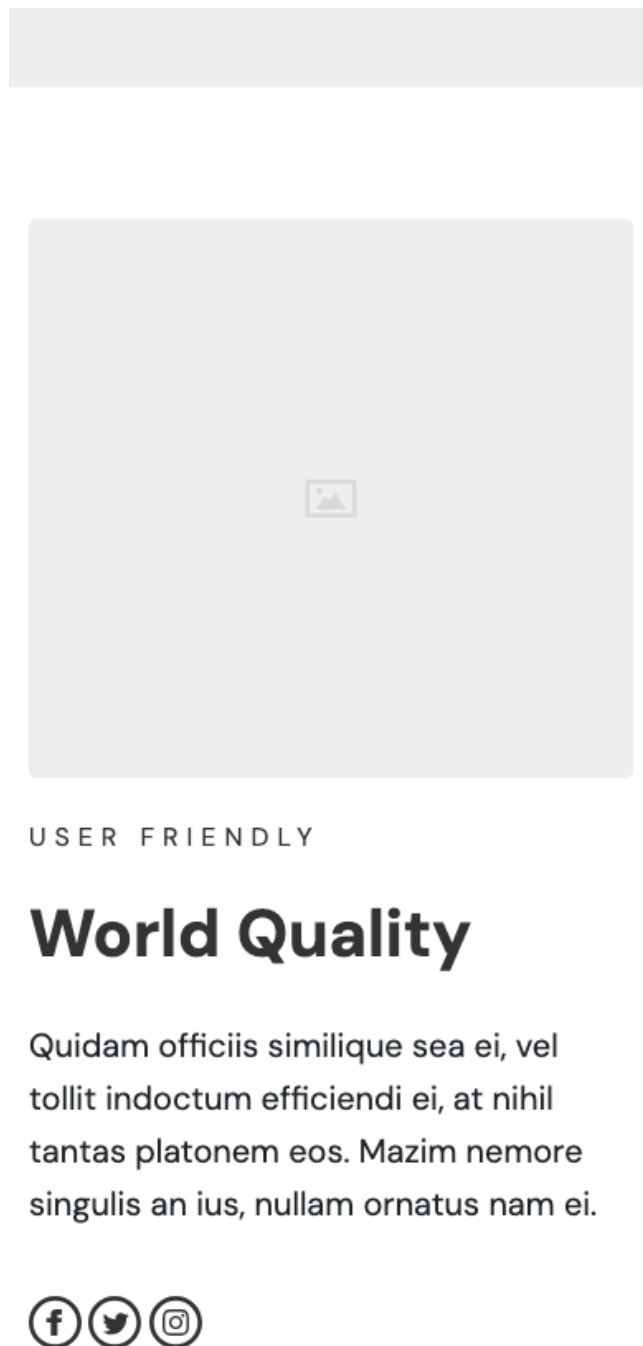


Рисунок 8 — Лендинг на мобильной ширине (контент)

3.2 Генератор паролей

Password Generator формирует случайный пароль заданной длины. Доступны кириллица, заглавные, цифры и спецсимволы; результат копируется в буфер; настройки открываются в dialog. Алгоритм: `getSettings` → `getAlphabet` → `getPassword`. Оценка: 10 из 10.

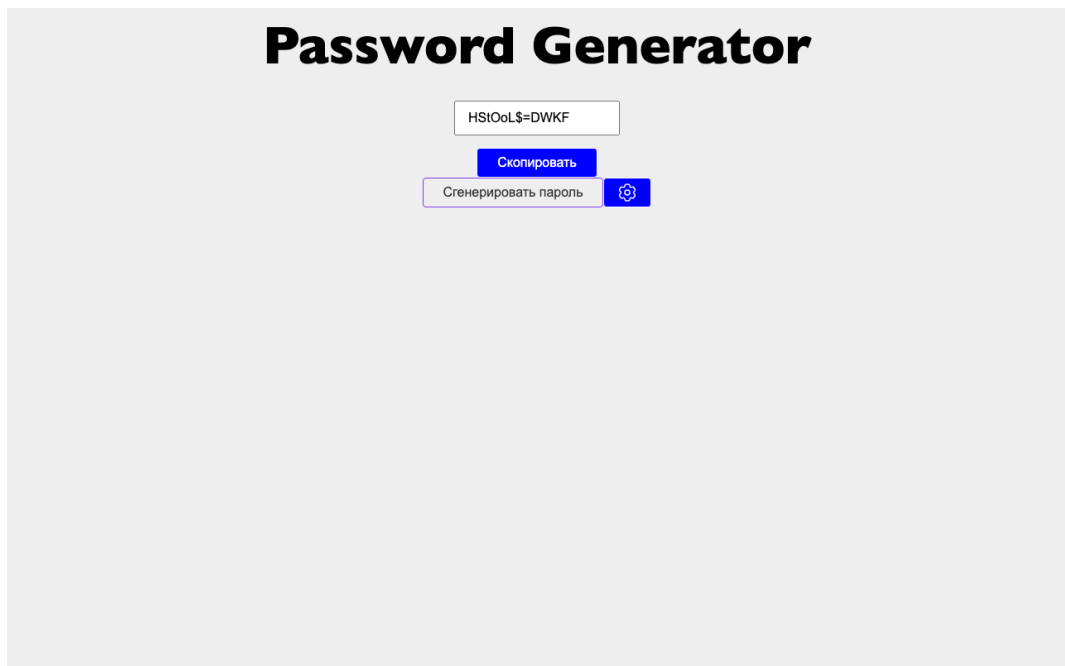


Рисунок 9 — Интерфейс генератора паролей

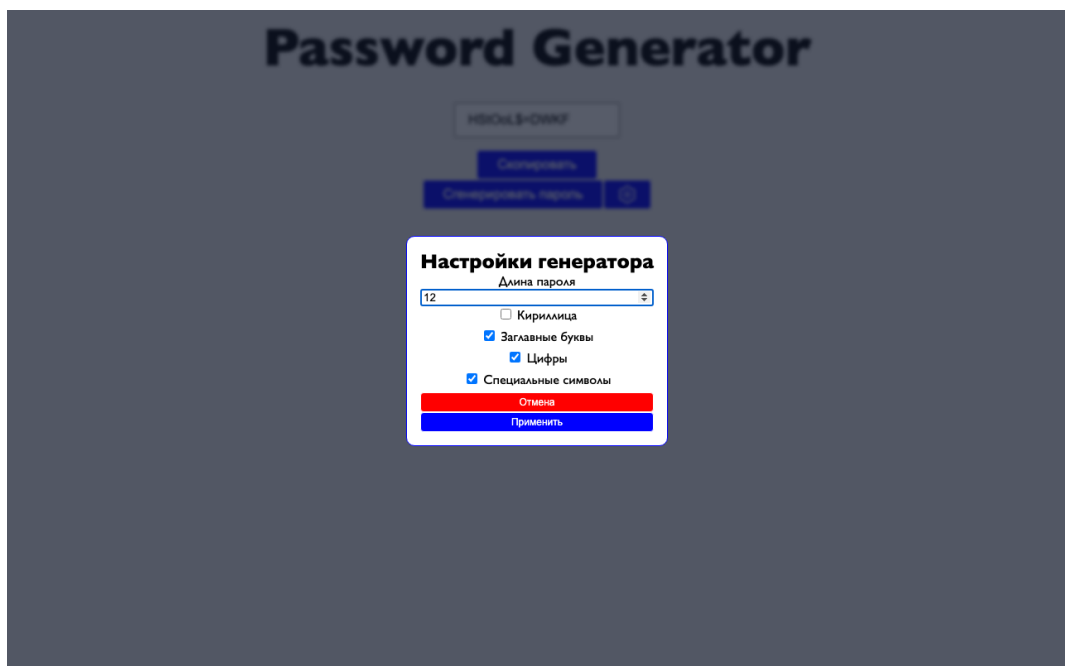


Рисунок 10 — Модальное окно настроек

3.3 Игра «Блэкджек»

BlackJack 21 — игра с банком, ставками и действиями Hit/Stand/Double. Логика в 21.js, UI в index.js. Колода 52 карты, shuffle Фишера–Йетса, учёт тузов, добор дилера до 17. Оценка: 10 из 10.

Карты дилера:	Ваша ставка: 50
Ваш счет:	Очки:
Ваши карты:	Ваша ставка:
	Очки:
Раздать Удвоить Еще Хватит	

Рисунок 11 — Раунд блэкджека после раздачи

Карты дилера:	Ваша ставка: 50
Ваш счет:	Очки:
Ваши карты:	Ваша ставка:
	Очки:
Раздать Удвоить Еще Хватит	

Рисунок 12 — Состояние стола в ходе раунда

3.4 Разделитель тысяч

Алгоритмическое задание: функция `тысячи_разделителей` группирует цифры пробелами без `toLocaleString`. Оценка: 9 из 10 (единственное задание без максимума).

Разделитель тысяч

Результаты вызова `тысячи_разделителей`:

```
тысячи_разделителей(1000) → 1000  
тысячи_разделителей(10000, 23) → 10 000,23  
тысячи_разделителей(100000) → 100 000
```

Рисунок 13 — Демонстрация разделителя тысяч

3.5 Генератор случайных цветов фона

Background Changer каждые 1,5 с меняет HEX-цвет фона и заголовка через `setInterval`. Оценка: 10 из 10.

Background Changer

Рисунок 14 — Смена цвета фона (кадр 1)



Рисунок 15 — Смена цвета фона (кадр 2)

3.6 Список задач (To-Do List)

To Do List: статусы active/completed/deleted, фильтр, localStorage, escapeHtml, делегирование событий. Архитектура state → render → saveState.
Оценка: 10 из 10.



Рисунок 16 — Список активных задач

Рисунок 17 — Фильтр завершённых задач

3.7 Подбор цветов (колоночная палитра)

Color Generator — пять колонок HEX-цветов, генерация по Space, замок колонки, копирование кода, синхронизация с location.hash. Оценка: 10 из 10.

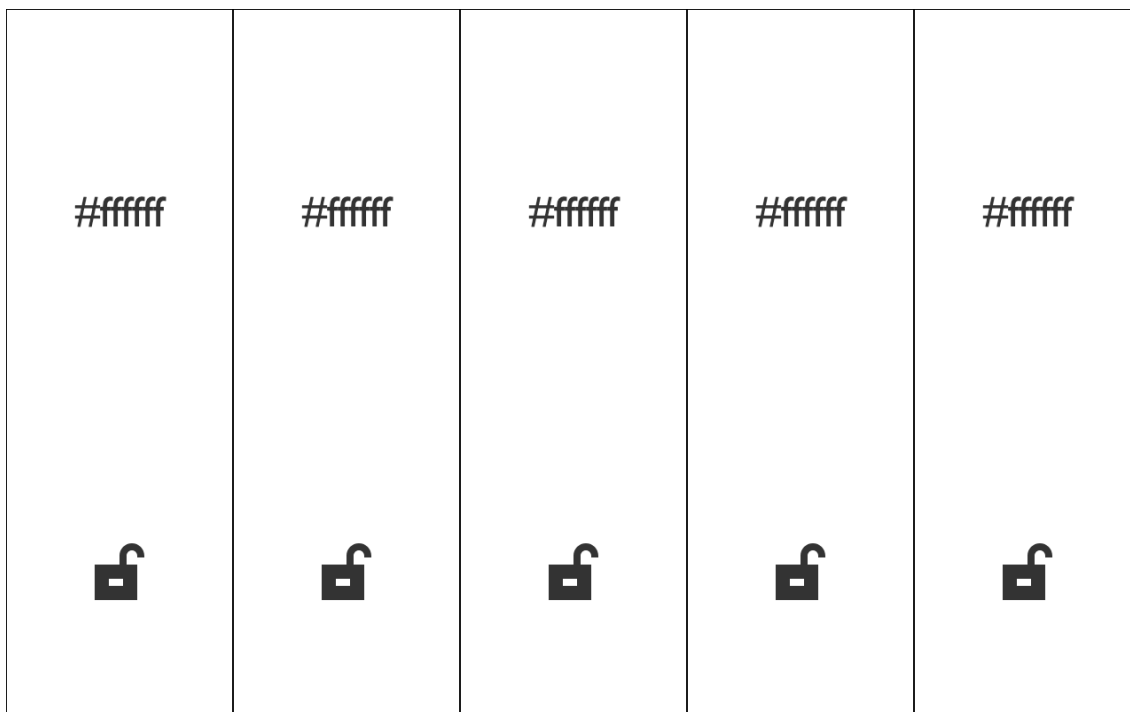


Рисунок 18 — Палитра из пяти колонок

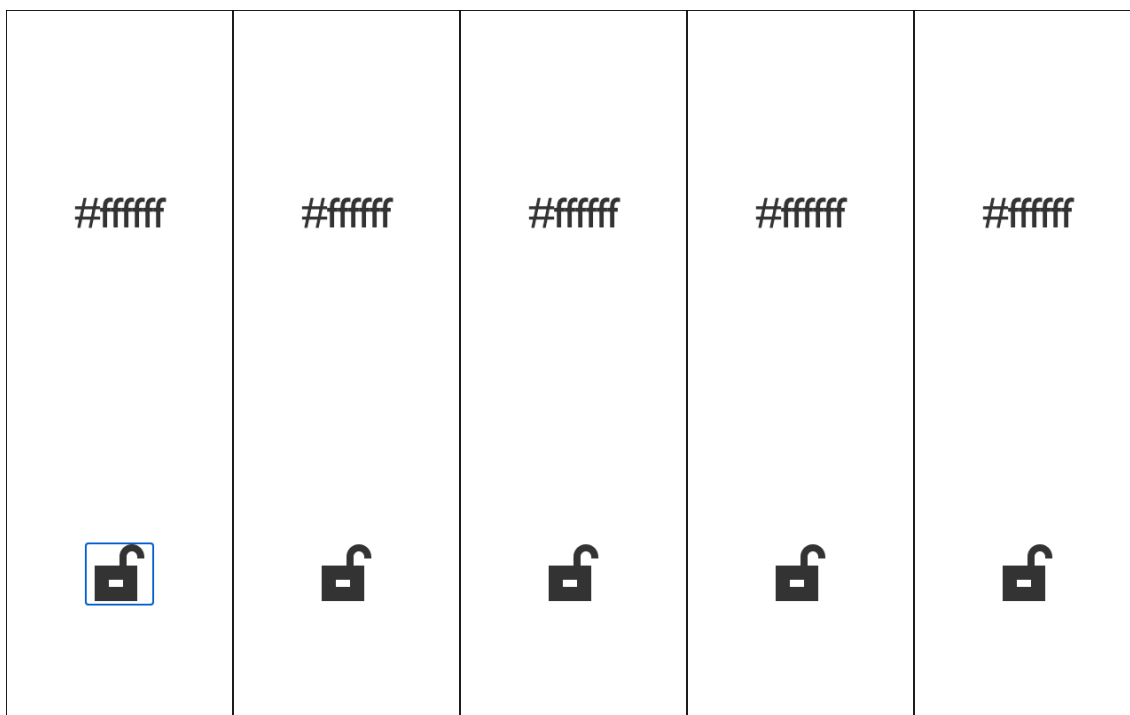


Рисунок 19 — Палитра после блокировки колонки

Совокупность заданий образует траекторию от статичной вёрстки к алгоритмам и приложениям с состоянием. Листинги ключевых функций — в приложении А.

4 Результаты практики

4.1 Итоги выполнения заданий

Сформирован комплект из семи проектов. Средний балл $\approx 9,86$, что соответствует оценке «отлично». Сводные результаты — в таблице 1.

Таблица 1 — Результаты приёмки практических заданий

№	Проект	Ключевые технологии	Оценка
1	Лендинг Powered Media	HTML, CSS, BEM, Bootstrap	10
2	Генератор паролей	JS, dialog, Clipboard API	10
3	Блэkdжек	ES-модули, DOM, алгоритмы	10
4	Разделитель тысяч	JS (строки, массивы)	9
5	Смена цвета фона	setInterval, style	10
6	To-Do List	state/render, localStorage	10
7	Палитра цветов	Grid, hash, Clipboard	10

Репозиторий `norbit-internship` фиксирует прогресс и может использоваться как портфолио учебных фронтенд-работ.

4.2 Освоенные навыки и методические замечания

Освоены: семантический HTML; Flexbox/Grid; BEM; Bootstrap 5; ES-модули; события и делегирование; алгоритмы; Clipboard, localStorage, hash; state/render; базовая защита от XSS.

Цикл работы: анализ → структура файлов → вёрстка → логика → тест в браузере → правки. Снижение балла по разделителю тысяч подчеркнуло важность граничных условий.

Ограничение программы — отсутствие обязательного бэкенда и автотестов; далее полезны HTTP API, сборщики и unit-тесты чистых функций.

4.3 Сравнительный анализ проектов

Лендинг максимален по CSS; блэджек и To-Do — по состоянию; утилиты закрепляют API; разделитель тысяч — чистый алгоритм. Паттерны BEM, clipboard и state/render переиспользовались между заданиями.

Индивидуальное задание выполнено: теория закреплена практикой, получены измеримые оценки, подготовлены иллюстрации и листинги кода.

ЗАКЛЮЧЕНИЕ

В ходе практики в организации «Норбит» (г. Санкт-Петербург) с 04.08.2026 по 28.08.2026 изучены основы фронтенд-разработки и выполнен комплекс заданий на HTML, CSS и JavaScript.

Рассмотрены сведения о компании-интеграторе и принципы клиентской веб-разработки. Практический результат: лендинг, генератор паролей, блэkdжек, разделитель тысяч, смена фона, To-Do List, палитра. Оценки: шесть проектов 10/10, один 9/10.

Цель практики достигнута, задачи индивидуального задания выполнены. Полученные навыки служат основой для дальнейшего изучения современной веб-разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления. – М. : Стандартинформ, 2017.
2. ГОСТ 2.105-2019. Общие требования к текстовым документам. – М. : Стандартинформ, 2019.
3. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. – М. : Стандартинформ, 2018.
4. Для учебного процесса : нормативная документация ГУАП. – URL: <https://guap.ru/c/regdocs/docs/uch> (дата обращения: 11.08.2026).
5. НОРБИТ : официальный сайт. – URL: <https://www.norbit.ru/> (дата обращения: 11.08.2026).
6. MDN Web Docs. HTML. – URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 20.08.2026).
7. MDN Web Docs. CSS. – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения: 20.08.2026).
8. MDN Web Docs. JavaScript. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 22.08.2026).
9. MDN Web Docs. Document Object Model (DOM). – URL: https://developer.mozilla.org/ru/docs/Web/API/Document_Object_Model (дата обращения: 22.08.2026).
10. MDN Web Docs. Window: localStorage. – URL: <https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage> (дата обращения: 24.08.2026).
11. Get BEM. Quick Start. – URL: <https://getbem.com/introduction/> (дата обращения: 18.08.2026).
12. Bootstrap 5 Documentation. – URL: <https://getbootstrap.com/docs/5.3/getting-started/introduction/> (дата обращения: 18.08.2026).

13. Учебные материалы программы фронтенд-разработки / ООО «Норбит». – Санкт-Петербург, 2026.

14. Репозиторий практических заданий norbit-internship. – URL: <https://github.com/skv0r/norbit-internship> (дата обращения: 28.08.2026).

ПРИЛОЖЕНИЕ А

Листинги ключевых фрагментов кода

В приложении приведены ключевые фрагменты исходного кода выполненных проектов с краткими поясняющими комментариями (синтаксис //). Листинги иллюстрируют решения, описанные в разделе 3: стили лендинга, генерацию паролей, логику блэkdжека, форматирование чисел, смену цвета, To-Do List и палитру цветов.

Листинг А.1 — CSS лендинга: переменные и блок header (BEM)

```
:root {
  --text-color: #333333; // основной цвет текста
  --bg-color: #eee;      // фон страницы
}

.header {
  padding-top: 16px;      // отступы шапки
  padding-bottom: 16px;
}

.header__logo {
  font-weight: 700;       // элемент логотипа блока header
  font-size: 40px;
}

.header__menu-item {
  font-size: 16px;
  font-weight: 700;
  color: var(--text-color); // использование CSS-переменной
}

.header__burger {
  display: none;          // на десктопе скрыт, в media.css включается
  font-size: 36px;
}

.banner__title {
  // крупный заголовок баннера лендинга
  font-size: 72px;
  font-weight: 700;
  text-transform: uppercase;
}
```

Листинг А.2 — CSS лендинга: карточки тарифов

```
.plans__card {
  // базовая карточка тарифного плана
  display: flex;
  flex-direction: column;
  padding: 32px;
  border-radius: 8px;
  background: #fff;
}
```

```

.plans__card--featured {
  // модификатор: выделенный (рекомендуемый) тариф
  background: #111;
  color: #fff;
}

.plans__price {
  font-size: 48px;          // крупная цена
  font-weight: 700;
}

.plans__feature {
  // пункт списка возможностей тарифа
  margin-bottom: 12px;
  line-height: 1.5;
}

```

Листинг А.3 — Генератор паролей: сбор алфавита и генерация

```

const LOWER_LATIN = "abcdefghijklmnopqrstuvwxyz";
const UPPER_LATIN = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
const LOWER_CYR = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя";
const DIGITS = "0123456789";
const SPECIAL = "!@#$$%^&*()-_+=[]{};:,.?";

function getRandomSymbol(alphabet) {
  // один случайный символ из строки алфавита
  return alphabet[Math.floor(Math.random() * alphabet.length)];
}

function getSettings() {
  // читаем длину и флаги чекбоксов из формы настроек
  return {
    length: document.getElementById("password-length").value,
    isCyrillic: document.getElementById("cyrillic").checked,
    isUppercase: document.getElementById("uppercase").checked,
    isDigits: document.getElementById("digits").checked,
    isSpecial: document.getElementById("special").checked,
  };
}

function getAlphabet(s) {
  // собираем алфавит по выбранным опциям
  let alphabet = s.isUppercase ? LOWER_LATIN + UPPER_LATIN : LOWER_LATIN;
  if (s.isCyrillic) {
    alphabet += s.isUppercase ? LOWER_CYR + UPPER_CYR : LOWER_CYR;
  }
  if (s.isDigits) alphabet += DIGITS;
  if (s.isSpecial) alphabet += SPECIAL;
  return alphabet;
}

function getPassword() {
  const s = getSettings();
  const len = Number(s.length);
  const alphabet = getAlphabet(s);
  if (!alphabet.length || len < 1) return ""; // нет символов — пустой
  результат

  let password = "";
  for (let i = 0; i < len; i++) {
    password += getRandomSymbol(alphabet); // наращиваем пароль
  }
}

```

```

    }
    return password;
}

function applyGeneratedPassword() {
    const pwd = getPassword();
    if (!pwd) {
        alert("Выберите хотя бы один тип символов и задайте длину.");
        return;
    }
    passwordInput.value = pwd; // выводим в поле интерфейса
}

copyBtn.addEventListener("click", () => {
    const text = passwordInput.value.trim();
    if (!text) return;
    navigator.clipboard.writeText(text) // Clipboard API
        .then(() => alert("Скопировано"))
        .catch(() => alert("Не удалось скопировать"));
});

```

Листинг А.4 — Блэкджек: колода, shuffle и подсчёт очков

```

const rank = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"];
const suits = ["♠", "♥", "♦", "♣"];

export function createDeck() {
    // создаём полную колоду 52 карты
    const deck = [];
    for (let i = 0; i < rank.length; i++) {
        const r = rank[i];
        // туз = 11, картинки = 10, иначе число
        let cardValue = r === "A" ? 11 : (Number(r) || 10);
        for (let j = 0; j < suits.length; j++) {
            deck.push({ rank: r, suit: suits[j], value: cardValue });
        }
    }
    return deck;
}

export function shuffleDeck(deck) {
    // перемешивание Фишера-Йетса
    for (let i = 0; i < deck.length; i++) {
        const j = Math.floor(Math.random() * (i + 1));
        [deck[i], deck[j]] = [deck[j], deck[i]];
    }
    return deck;
}

export function getHandValue(hand) {
    // сумма очков; тузы понижаем с 11 до 1 при переборе
    let total = hand.reduce((sum, card) => sum + card.value, 0);
    let aceCount = hand.filter((card) => card.rank === "A").length;
    while (total > 21 && aceCount > 0) {
        total -= 10;
        aceCount--;
    }
    return total;
}

```

Листинг А.5 — Блэкджек: старт раунда, Hit, Stand, Double

```

export function startRound() {

```

```

// старт, если раунд не активен и хватает банка на ставку
if (!state.isRoundActive && state.bank >= state.bet) {
  state.deck = shuffleDeck(createDeck());
  state.dealerCards = [];
  state.playerCards = [];
  state.currentBet = state.bet;
  state.result = "";

  state.dealerCards.push(getCard()); // дилеру одна карта
  state.playerCards.push(getCard()); // игроку две
  state.playerCards.push(getCard());
  state.bank -= state.currentBet; // списываем ставку
  state.isRoundActive = true;
}
}

export function Hit() {
  if (!state.isRoundActive) return;
  state.playerCards.push(getCard()); // взять карту
  if (getHandValue(state.playerCards) > 21) {
    finishRound(); // перебор — конец раунда
  }
}

export function Stand() {
  if (!state.isRoundActive) return;
  // дилер добирает до 17
  while (getHandValue(state.dealerCards) < 17) {
    state.dealerCards.push(getCard());
  }
  finishRound();
}

export function Double() {
  // удвоение только на двух картах и при достаточном банке
  if (!state.isRoundActive) return;
  if (state.playerCards.length !== 2) return;
  if (state.bank < state.currentBet) return;

  state.bank -= state.currentBet;
  state.currentBet *= 2;
  Hit();
  if (getHandValue(state.playerCards) <= 21) Stand();
}

function finishRound() {
  const playerTotal = getHandValue(state.playerCards);
  const dealerTotal = getHandValue(state.dealerCards);

  if (playerTotal > 21) {
    state.result = "Перебор! Вы проиграли.";
  } else if (dealerTotal > 21 || playerTotal > dealerTotal) {
    state.result = "Вы выиграли!";
    state.bank += state.currentBet * 2; // выплата 1:1
  } else if (playerTotal === dealerTotal) {
    state.result = "Ничья.";
    state.bank += state.currentBet; // возврат ставки
  } else {
    state.result = "Дилер выиграл.";
  }
  state.currentBet = 0;
  state.isRoundActive = false;
}

```

Листинг А.6 — Разделитель тысяч

```
function тысячи_разделителей(n, d) {
  // целая часть без знака
  let s = String(Math.abs(Math.trunc(n)));

  if (s.length > 4) {
    // группируем по 3 цифры справа налево
    const p = [];
    while (s.length > 3) {
      p.unshift(s.slice(-3));
      s = s.slice(0, -3);
    }
    p.unshift(s);
    s = p.join(" "); // пробел как разделитель тысяч
  }

  // знак минус + опциональная дробная часть через запятую
  return (n < 0 ? "-" : "") + s + (d != null ? "," + d : "");
}

console.log(тысячи_разделителей(1000));           // 1000
console.log(тысячи_разделителей(10000, 23));      // 10 000,23
console.log(тысячи_разделителей(100000));         // 100 000
```

Листинг А.7 — Смена цвета фона

```
const title = document.getElementById("title");

function getRandomColor() {
  // HEX-цвет вида #a1b2c3
  return "#" + Math.floor(Math.random() * 16777215)
    .toString(16)
    .padStart(6, "0");
}

function setNewBackground() {
  document.body.style.backgroundColor = getRandomColor();
}

function setNewTextColor() {
  title.style.color = getRandomColor();
}

// обновляем фон и цвет заголовка каждые 1.5 с
setInterval(() => {
  setNewBackground();
  setNewTextColor();
}, 1500);
```

Листинг А.8 — To-Do List: состояние, render, localStorage

```
const STORAGE_KEY = "todo-list-state";
const state = { tasks: [], filter: "active" };

function escapeHtml(text) {
  // защита от XSS при вставке текста в HTML
  return String(text)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/"/g, "&quot;");
}
```

```

function getFilteredTasks(state) {
  // оставляем задачи текущего фильтра
  return state.tasks.reduce((acc, task) => {
    if (state.filter === task.status) acc.push(task);
    return acc;
  }, []);
}

function render(tasks) {
  // перерисовываем список по массиву задач
  let html = "";
  tasks.forEach((task) => {
    const safeText = escapeHtml(task.text);
    html += `<li class="todo__task todo__task--${task.status}"
      data-task-id="${task.id}">
      <p class="todo__task-text">${safeText}</p>
      <div class="todo__task-actions">...</div>
    </li>`;
  });
  tasksList.innerHTML = html;
}

function addTask(text) {
  const trimmed = String(text).trim();
  if (!trimmed) return;
  state.tasks.push({ id: Date.now(), text: trimmed, status: "active" });
  state.filter = "active";
  render(getFilteredTasks(state));
  saveState(); // сразу сохраняем
}

function saveState() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify({
      tasks: state.tasks,
      filter: state.filter,
    }));
  } catch { /* storage недоступен */ }
}

function loadState() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return false;
    const data = JSON.parse(raw);
    if (!data || !Array.isArray(data.tasks)) return false;
    state.tasks = data.tasks;
    state.filter = data.filter || "active";
    return true;
  } catch {
    return false;
  }
}

```

Листинг А.9 — To-Do List: делегирование событий и статусы

```

function completeTask(task) { task.status = "completed"; }
function moveToTrash(task) {
  if (task.status === "active" || task.status === "completed") {
    task.status = "deleted";
  }
}

```

```

function restoreTask(task) {
  if (task.status === "deleted" || task.status === "completed") {
    task.status = "active";
  }
}
function deleteTask(taskId) {
  // окончательное удаление из массива
  state.tasks = state.tasks.filter((t) => t.id !== taskId);
}

// один обработчик на весь список (event delegation)
tasksList.addEventListener("click", (e) => {
  const row = e.target.closest(".todo__task");
  if (!row) return;
  const taskItem = findTaskById(Number(row.dataset.taskId));
  if (!taskItem) return;

  if (e.target.closest(".todo__btn--approve")) {
    if (taskItem.status !== "active") return;
    completeTask(taskItem);
  } else if (e.target.closest(".todo__btn--del")) {
    if (taskItem.status === "deleted") deleteTask(taskItem.id);
    else moveToTrash(taskItem);
  } else if (e.target.closest(".todo__btn--restore")) {
    restoreTask(taskItem);
  } else return;

  render(getFilteredTasks(state));
  saveState();
});

```

Листинг А.10 — Палитра цветов: генерация, hash, lock, clipboard

```

function getRandomColor() {
  return "#" + Math.floor(Math.random() * 16777215)
    .toString(16).padStart(6, "0");
}

function changeBackgroundColor(col) {
  if (col.classList.contains("locked")) return; // замок — не меняем
  const color = getRandomColor();
  col.style.backgroundColor = color;
  const hexEl = col.querySelector(".hex");
  if (hexEl) hexEl.textContent = color;
}

function updateHash() {
  // пишем палитру в URL: #aabbcc-ddeeff-...
  const colors = [...columns].map((col) =>
    col.querySelector(".hex").textContent.trim().substring(1)
  );
  location.hash = colors.join("-");
}

function applyHashOnLoad() {
  // восстанавливаем цвета из hash при открытии ссылки
  const hash = location.hash.replace("#", "");
  if (!hash) return;
  const colors = hash.split("-");
  columns.forEach((col, i) => {
    if (!colors[i]) return;
    const color = `#${colors[i]}`;
    col.style.backgroundColor = color;
  });
}

```

```

        col.querySelector(".hex").textContent = color;
    });
}

function showToast(message) {
    const toast = document.createElement("div");
    toast.textContent = message;
    toast.className = "toast";
    document.body.appendChild(toast);
    setTimeout(() => toast.remove(), 1000);
}

colButton.addEventListener("click", (e) => {
    e.stopPropagation();
    const isLocked = col.classList.toggle("locked"); // вкл/выкл замок
    colButton.innerHTML = isLocked ? LOCK_CLOSED_SVG : LOCK_OPEN_SVG;
});

colText.addEventListener("click", async (e) => {
    e.stopPropagation();
    try {
        await navigator.clipboard.writeText(colText.textContent.trim());
        showToast("Текст успешно скопирован");
    } catch {
        showToast("Ошибка при копировании");
    }
});

applyHashOnLoad();

```

Листинг А.11 — Адаптив лендинга (фрагмент media.css)

```

@media (max-width: 992px) {
    .header__menu {
        display: none;          // скрываем горизонтальное меню
    }
    .header__burger {
        display: inline-block;  // показываем кнопку «бургер»
    }
    .h2-title {
        font-size: 40px;        // уменьшаем заголовки секций
        line-height: 48px;
    }
}

@media (max-width: 768px) {
    .plans {
        // тарифы в одну колонку на узком экране
        display: flex;
        flex-direction: column;
        gap: 16px;
    }
    .banner__title {
        font-size: 40px;
    }
}

```

Листинг А.12 — Блэджек: HTML-карточка и состояние

```

let state = {
    deck: [],
    playerCards: [],
    dealerCards: [],

```



```

    bet: 20,
    currentBet: 0,
    bank: 1000,           // стартовый банк
    isRoundActive: false, // блокирует/разрешает кнопки UI
    result: ""
  };

export function renderCard(card) {
  // разметка одной карты для вставки в DOM
  return `
    <div class="card">
      <span class="card-corner card-corner--top">${card.rank}</span>
      <span class="card-suit">${card.suit}</span>
      <span class="card-corner card-corner--bottom">${card.rank}</span>
    </div>`;
}

export function setBet(value) {
  // валидация ставки перед стартом раунда
  const nextBet = Number(value);
  if (!Number.isFinite(nextBet) || nextBet < 1) return;
  state.bet = Math.floor(nextBet);
}

function getCard() {
  return state.deck.pop(); // взять верхнюю карту колоды
}

```